

利用適用於 Flutter 的質感設計動態效果建構精美的轉換效果

來源：<https://codelabs.developers.google.com/codelabs/material-motion-flutter?hl=zh-tw>

步驟數：12

使用動畫套件中的轉場效果，將 Material 的動態系統加入 Reply 應用程式。

目錄

1. 簡介
2. 設定 Flutter 開發環境
3. 下載程式碼實驗室範例應用程式
4. 熟悉範例應用程式程式碼
5. 從電子郵件名單新增容器轉換轉場至電子郵件詳細資料頁面
6. 從 FAB 新增容器轉換轉場效果至撰寫電子郵件頁面
7. 從搜尋圖示到搜尋檢視畫面頁面，新增共用 Z 軸轉換
8. 在信箱頁面之間新增「淡出淡入」轉場效果
9. 在撰寫和回覆懸浮動作按鈕之間新增「淡入/淡出」轉場效果
10. 在消失的信箱標題之間新增「淡出」轉場效果
11. 在底部應用程式列動作之間新增「淡出淡入」轉場效果
12. 恭喜！

步驟 1 · 約 0 分鐘

1. 簡介

Material Design 是一套系統，可協助您打造大膽且美觀的數位產品。只要在一致的原則和元件下整合樣式、品牌、互動和動態效果，產品團隊就能發揮最大的設計潛力。



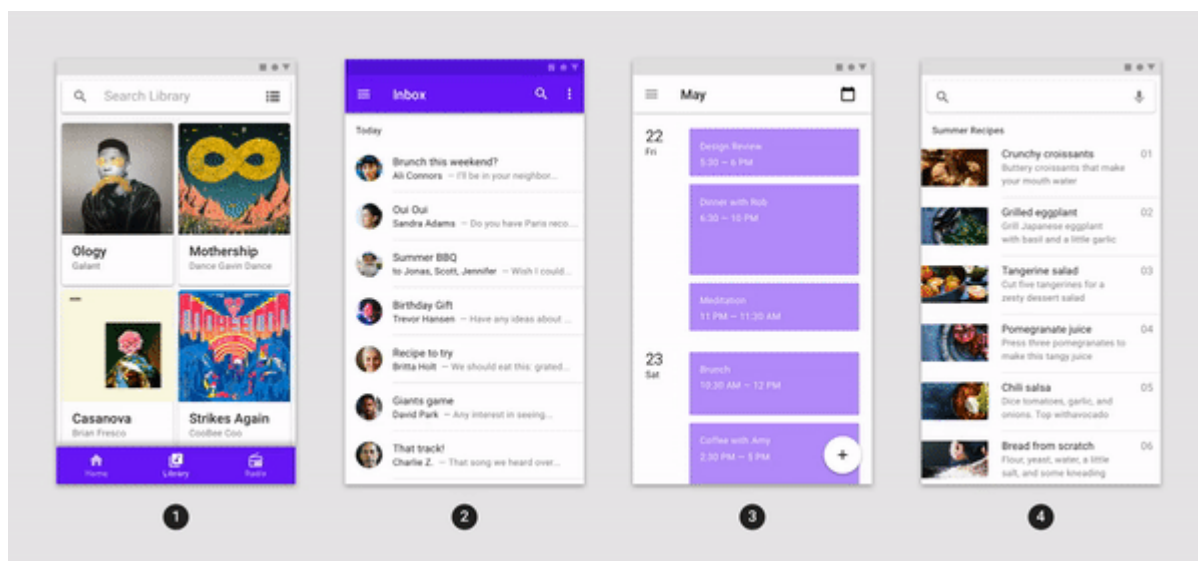
開發人員可透過 Material 元件 (MDC) 實作 Material Design。MDC 由 Google 的工程師和 UX 設計師團隊打造，提供數十種美觀實用的 UI 元件，適用於 Android、iOS、網頁和 Flutter。material.io/develop

什麼是適用於 Flutter 的 Material 動態效果系統？

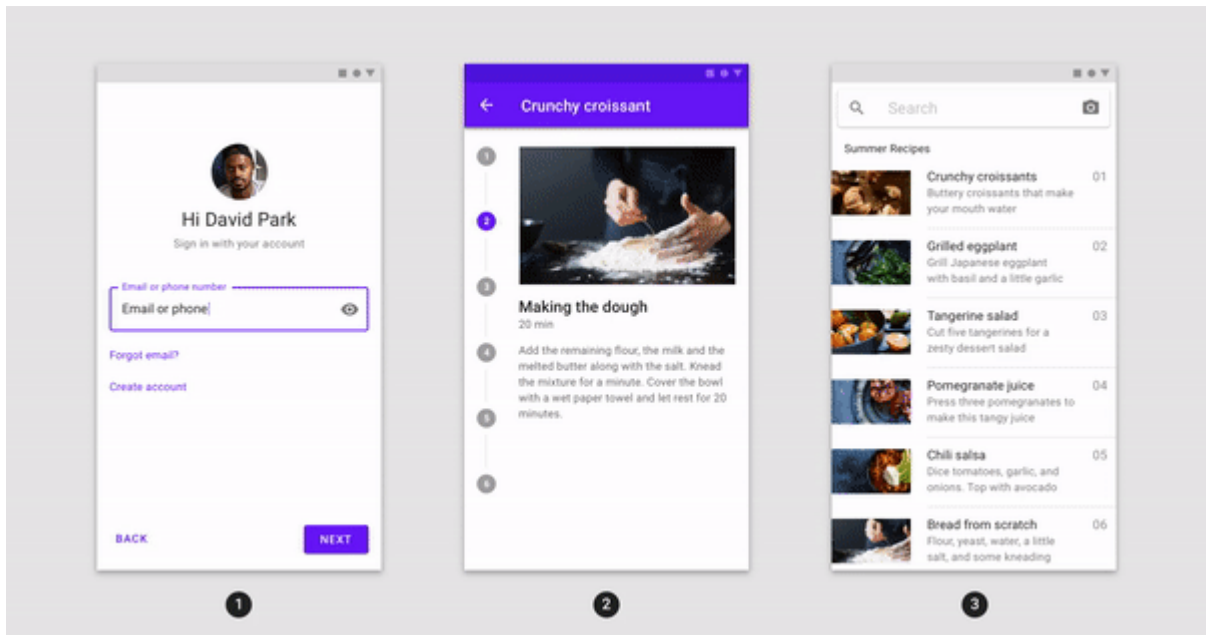
Flutter 的 **Material Design** 動態效果系統是一組動畫套件中的轉場模式，可協助使用者瞭解及瀏覽應用程式，如 [Material Design 規範](#) 所述。

Material 轉換主要有四種模式，分別是：

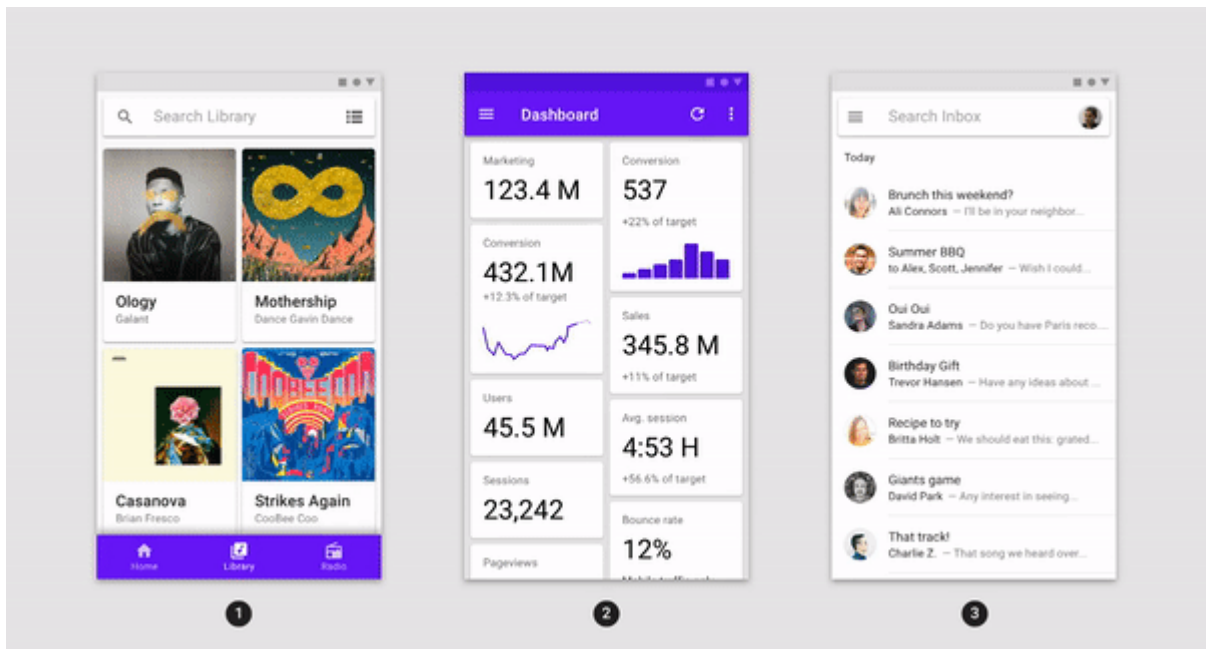
- **容器轉換**：轉換含有容器的 UI 元素，並將一個元素無縫轉換為另一個元素，在兩個不同的 UI 元素之間建立可見的連結。



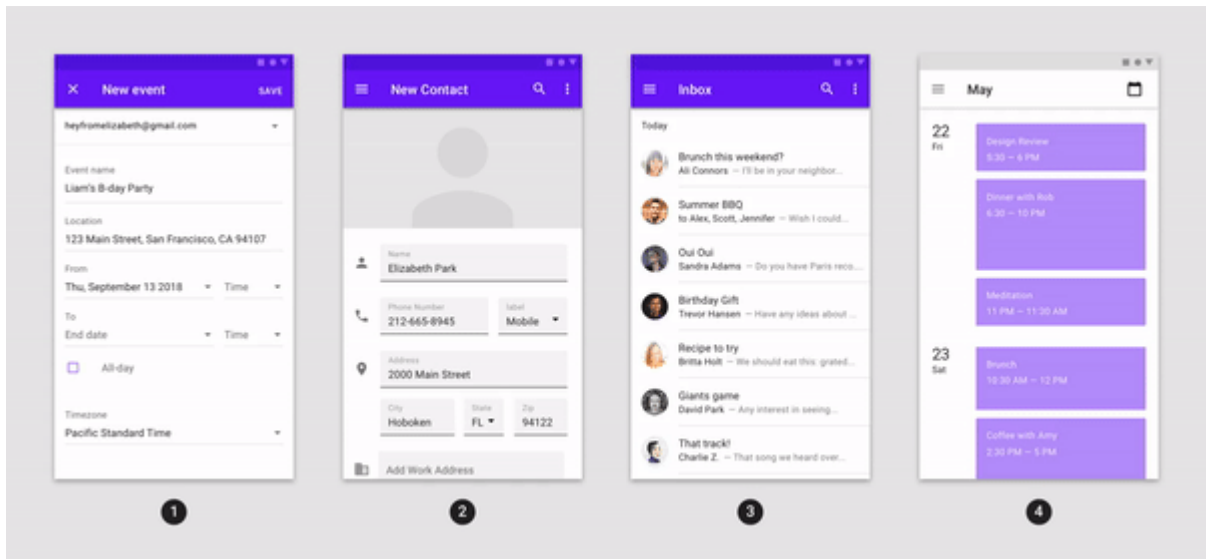
- **共用軸**：轉換 UI 元素，這些 UI 元素彼此間具有空間或導覽的關係；轉換時共用 X、Y 或 Z 軸，藉此強調元素間的關係。



- 淡出淡入：轉換彼此間沒有緊密關係的 UI 元素；使用連續淡出和淡入效果，並縮放進入的元素。



- 淡化：用於進入或退出畫面範圍的 UI 元素。



動畫套件提供這些模式的轉場小工具，這些小工具是以 [Flutter 動畫程式庫](#) (`flutter/animation.dart`) 和 [Flutter Material 程式庫](#) (`flutter/material.dart`) 為基礎建構而成：

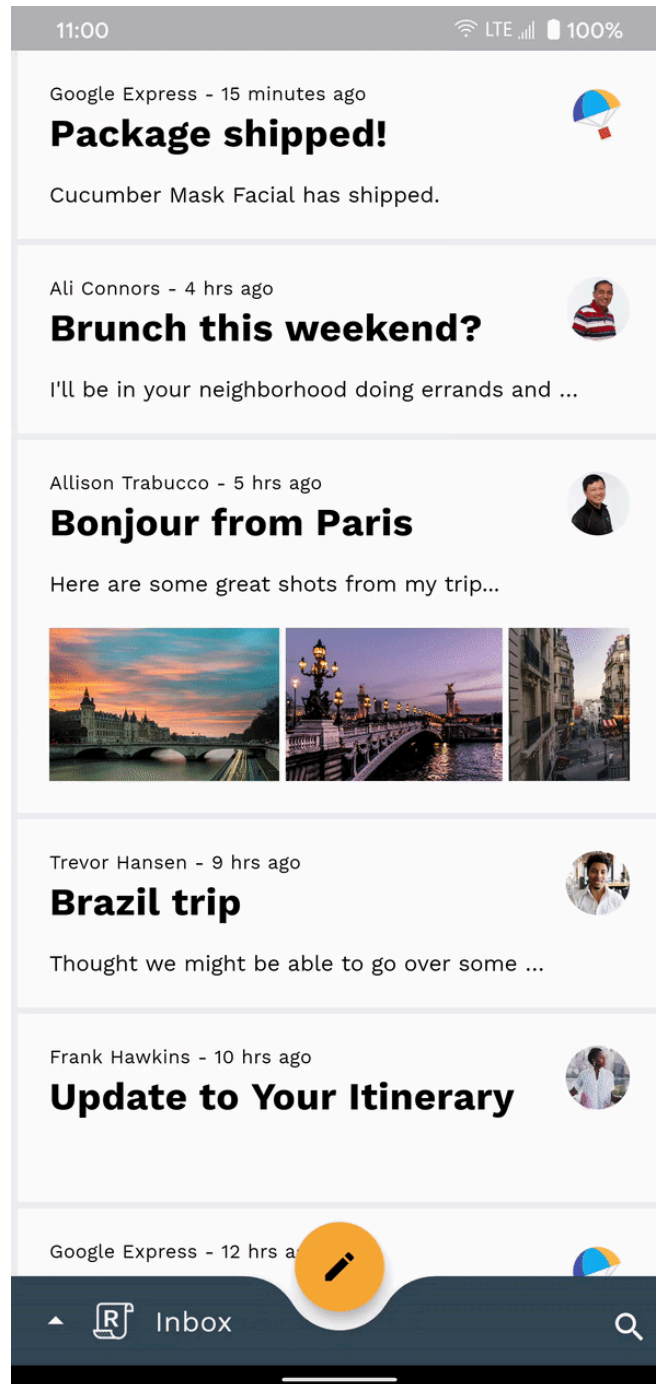
在本程式碼研究室中，您將使用以 Flutter 架構和 Material 程式庫為基礎建構的 Material 轉換效果，也就是處理小工具。:)

建構項目

本程式碼研究室會引導您使用 Dart，在名為「Reply」的範例 Flutter 電子郵件應用程式中建構一些轉場效果，示範如何使用動畫套件中的轉場效果，自訂應用程式的外觀和風格。

我們會提供 Reply 應用程式的範例程式碼，您將在應用程式中加入下列 Material 轉場效果，如下方完成的程式碼研究室 GIF 所示：

- 從電子郵件名單到電子郵件詳細資料頁面的**容器轉換**轉場效果
- **容器轉換**：從懸浮動作按鈕 (FAB) 轉換至撰寫電子郵件頁面
- **共用 Z 軸**：從搜尋圖示到搜尋檢視頁面的轉場效果
- 信箱頁面之間的**淡出**轉場效果
- Compose 和回覆 FAB 之間的「淡出」轉場效果
- 在消失的信箱標題之間使用「淡出」轉場效果
- 在底部應用程式列動作之間**淡出**轉場



在本程式碼研究室中，您將使用 [animations](#) 套件提供的轉場效果。如要進一步瞭解 Material Flutter 主題設定和元件系統，請參閱其他 [Material Design 程式碼研究室](#)。

軟硬體需求

- 具備 Flutter 開發和 Dart 的基本知識
- 程式碼編輯器
- Android/iOS 模擬器或裝置
- 程式碼範例 (請參閱下一個步驟)

2. 設定 Flutter 開發環境

如要完成本實驗室，您需要兩項軟體：[Flutter SDK](#) 和[編輯器](#)。

您可以使用下列任一裝置執行程式碼研究室：

- 連線至電腦並設為開發人員模式的實體 [Android](#) 或 [iOS](#) 裝置。
 - [iOS 模擬器](#) (需要安裝 Xcode 工具)。
 - [Android Emulator](#) (需在 Android Studio 中設定)。
 - 瀏覽器 (偵錯時必須使用 Chrome)。
 - 以 [Windows](#)、[Linux](#) 或 [macOS](#) 電腦版應用程式的形式。您必須在要部署的平台上進行開發。因此，如要開發 Windows 桌面應用程式，您必須在 Windows 上開發，才能存取適當的建構鏈。如需作業系統專屬需求，請參閱 docs.flutter.dev/desktop。
-

步驟 3 · 約 5 分鐘

3. 下載程式碼實驗室範例應用程式

選項 1：從 GitHub 複製入門程式碼研究室應用程式

如要從 GitHub 複製這個[程式碼研究室](#)，請執行下列指令：

```
git clone https://github.com/material-components/material-components-flutter-motion-codelab.git
cd material-components-flutter-motion-codelab
```

如需更多說明，請參閱「[從 GitHub 複製存放區](#)」。

選項 2：下載 入門程式碼研究室應用程式的 ZIP 檔案

範例應用程式位於 `material-components-flutter-motion-codelab-starter` 目錄中。

驗證專案依附元件

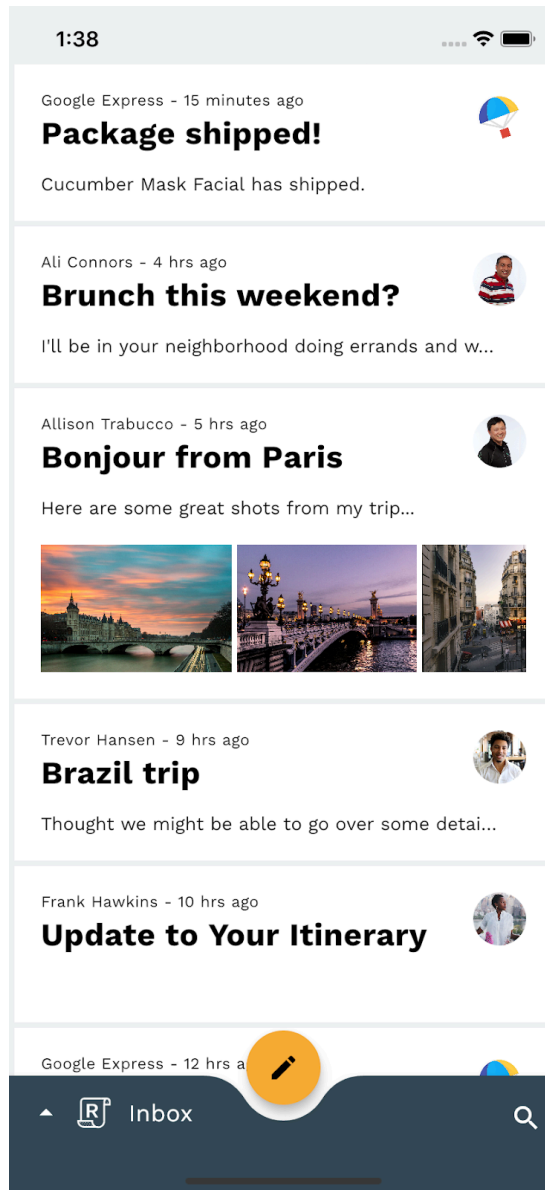
專案依附於 **animations** 套件。在 `pubspec.yaml` 中，請注意 `dependencies` 部分包含下列項目：

```
animations: ^2.0.0
```

開啟專案並執行應用程式

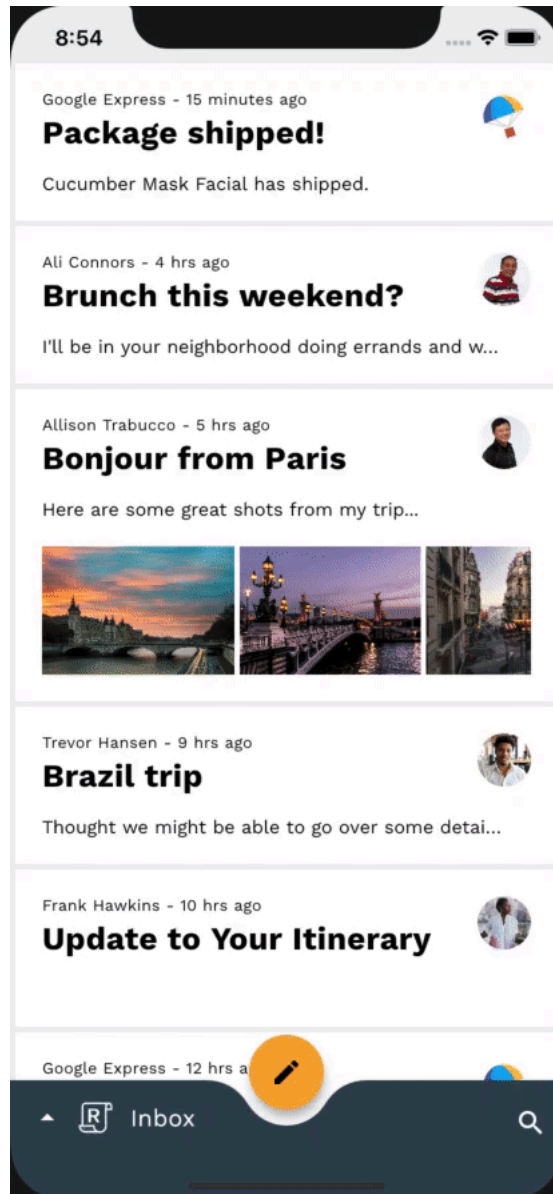
1. 在您選擇的編輯器中開啟專案。
2. 按照所選編輯器的「[開始使用：試用](#)」一節中的「執行應用程式」操作說明操作。

太棒了，Reply 首頁的範例程式碼應會在裝置/模擬器上執行。這時應該會看到收件匣，內含電子郵件清單。



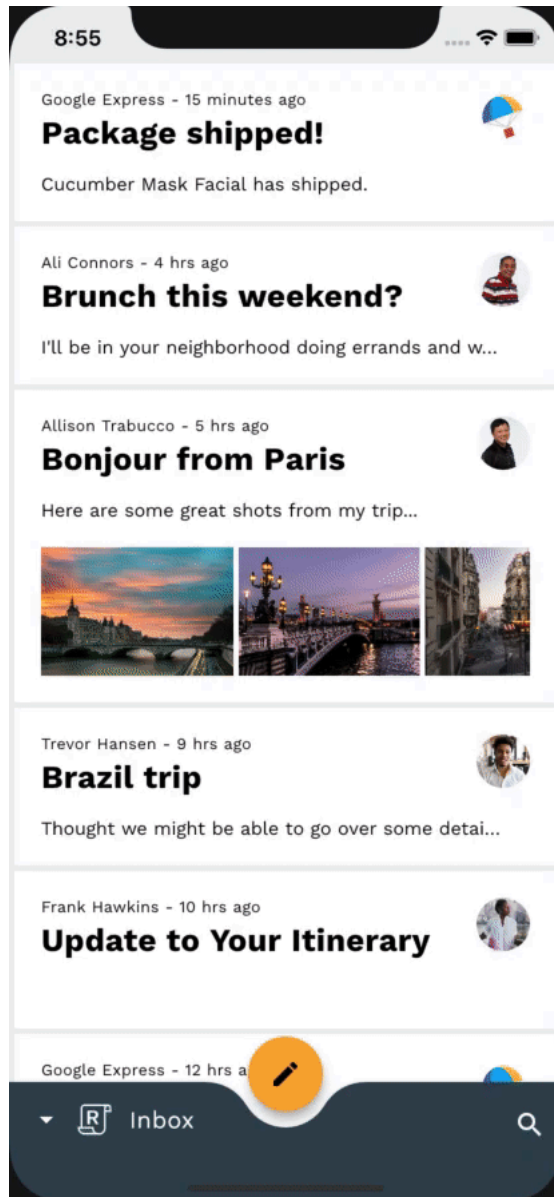
選用：放慢裝置動畫速度

由於本程式碼研究室涉及快速但精緻的轉場效果，因此在實作時，放慢裝置的動畫速度，有助於觀察轉場效果的細節。只要開啟底部導覽匣，然後輕觸設定圖示，即可透過應用程式內設定完成這項操作。請放心，這種減緩裝置動畫速度的方法不會影響 Reply 應用程式以外的裝置動畫。



選用：深色模式

如果 Reply 的亮色主題會造成眼睛不適，請繼續閱讀本文。應用程式內建設定，可將應用程式主題變更為深色模式，減輕眼睛負擔。開啟底部導覽匣時，輕觸設定圖示即可存取這項設定。



4. 熟悉範例應用程式程式碼

我們來看看程式碼。我們提供了一個應用程式，使用[動畫套件](#)在應用程式的不同畫面之間轉換。

- **首頁**：顯示所選信箱
- **InboxPage**：顯示電子郵件清單
- **MailPreviewCard**：顯示電子郵件的預覽畫面
- **MailViewPage**：顯示單一完整電子郵件
- **ComposePage**：可撰寫新電子郵件
- **SearchPage**：顯示搜尋檢視畫面

如要進一步瞭解本程式碼研究室中使用的相關概念，請參閱下列指南和練習：

- [編寫第一個 Flutter 應用程式](#)
- [讓 Flutter 應用程式從無趣變為美觀](#)
- [MDC-101 Flutter：Material Design 元件基本概念](#)
- [MDC-102 Flutter：Material Design 結構和版面配置](#)
- [MDC-103 Flutter：利用顏色、形狀、高度和類型建立 Material Design 主題](#)
- [MDC-104 Flutter：Material 進階元件](#)

router.dart

首先，如要瞭解應用程式的根層級導覽設定方式，請開啟 `lib` 目錄中的 `router.dart`：

```

class ReplyRouterDelegate extends RouterDelegate<ReplyRoutePath>
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<ReplyRoutePath> {
  ReplyRouterDelegate({required this.replyState})
    : navigatorKey = GlobalKey<NavigatorState>(replyState) {
    replyState.addListener(() {
      notifyListeners();
    });
  }

  @override
  final GlobalKey<NavigatorState> navigatorKey;

  RouterProvider replyState;

  @override
  void dispose() {
    replyState.removeListener(notifyListeners);
    super.dispose();
  }

  @override
  ReplyRoutePath get currentConfiguration => replyState.routePath!;

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider<RouterProvider>.value(value: replyState),
      ],
      child: Selector<RouterProvider, ReplyRoutePath?>(
        selector: (context, routerProvider) => routerProvider.routePath,
        builder: (context, routePath, child) {
          return Navigator(
            key: navigatorKey,
            onPopPage: _handlePopPage,
            pages: [
              // TODO: Add Shared Z-Axis transition from search icon to search view page
              CustomTransitionPage(
                transitionKey: ValueKey('Home'),
                screen: HomePage(),
              ),
              if (routePath is ReplySearchPath)
                CustomTransitionPage(
                  transitionKey: ValueKey('Search'),
                  screen: SearchPage(),
                ),
            ],
          );
        },
      ),
    );
  }
}

```

```

}

bool _handlePopPage(Route<dynamic> route, dynamic result) {
  // _handlePopPage should not be called on the home page because the
  // PopNavigatorRouterDelegateMixin will bubble up the pop to the
  // SystemNavigator if there is only one route in the navigator.
  assert(route.willHandlePopInternally ||
    replyState.routePath is ReplySearchPath);

  final bool didPop = route.didPop(result);
  if (didPop) replyState.routePath = const ReplyHomePath();
  return didPop;
}

@override
Future<void> setNewRoutePath(ReplyRoutePath configuration) {
  replyState.routePath = configuration;
  return SynchronousFuture<void>(null);
}
}

```

這是根層級的導覽器，可處理應用程式中會佔用整個畫布的畫面，例如 `HomePage` 和 `SearchPage`。這個函式會監聽應用程式的狀態，檢查我們是否已將路徑設為 `ReplySearchPath`。如果有的話，系統會重建導覽器，並將 `SearchPage` 放在堆疊頂端。請注意，我們的畫面會包裝在 `CustomTransitionPage` 中，且未定義任何轉場效果。這會顯示在畫面之間導覽的一種方式，不需要任何自訂轉場效果。

home.dart

我們在應用程式狀態中將路徑設為 `ReplySearchPath`，方法是在 `home.dart` 中執行下列操作：`_BottomAppBarActionItems`

```

Align(
  alignment: AlignmentDirectional.bottomEnd,
  child: IconButton(
    icon: const Icon(Icons.search),
    color: ReplyColors.white50,
    onPressed: () {
      Provider.of<RouterProvider>(
        context,
        listen: false,
      ).routePath = const ReplySearchPath();
    },
  ),
);

```

在 `onPressed` 參數中，我們會存取 `RouterProvider` 並將其 `routePath` 設為 `ReplySearchPath`。我們的 `RouterProvider` 會追蹤根導覽器的狀態。

mail_view_router.dart

現在來看看應用程式的內部導覽設定，在 `lib` 目錄中開啟 `mail_view_router.dart`。您會看到類似上方的導覽器：

```

class MailViewRouterDelegate extends RouterDelegate<void>
  with ChangeNotifier, PopNavigatorRouterDelegateMixin {
  MailViewRouterDelegate({required this.drawerController});

  final AnimationController drawerController;

  @override
  Widget build(BuildContext context) {
    bool _handlePopPage(Route<dynamic> route, dynamic result) {
      return false;
    }

    return Selector<EmailStore, String>(
      selector: (context, emailStore) => emailStore.currentlySelectedInbox,
      builder: (context, currentlySelectedInbox, child) {
        return Navigator(
          key: navigatorKey,
          onPopPage: _handlePopPage,
          pages: [
            // TODO: Add Fade through transition between mailbox pages (Motion)
            CustomTransitionPage(
              transitionKey: ValueKey(currentlySelectedInbox),
              screen: InboxPage(
                destination: currentlySelectedInbox,
              ),
            ),
          ],
        );
      },
    );
  }
  ...
}

```

這是我們的內在導航員。它會處理應用程式的內部畫面，這些畫面只會使用畫布主體，例如 `InboxPage`。 `InboxPage` 會根據應用程式狀態中的目前信箱，顯示電子郵件清單。每當應用程式狀態的 `currentlySelectedInbox` 屬性發生變更時，導覽器就會以堆疊頂端的正確 `InboxPage` 重建。

home.dart

我們在應用程式狀態中設定目前的信箱，方法是在 `home.dart` 內的 `_HomePageState` 中執行下列操作：

```

void _onDestinationSelected(String destination) {
  var emailStore = Provider.of<EmailStore>(
    context,
    listen: false,
  );

  if (emailStore.onMailView) {
    emailStore.currentlySelectedEmailId = -1;
  }

  if (emailStore.currentlySelectedInbox != destination) {
    emailStore.currentlySelectedInbox = destination;
  }

  setState(() {});
}

```

在 `_onDestinationSelected` 函式中，我們會存取 `EmailStore`，並將其 `currentlySelectedInbox` 設為所選目的地。我們的 `EmailStore` 會追蹤內部導覽器的狀態。

home.dart

最後，如要查看導覽路徑的用法範例，請開啟 `lib` 目錄中的 `home.dart`。找出 `_ReplyFabState` 類別，位於 `InkWell` 小工具的 `onTap` 屬性中，應如下所示：

```

onTap: () {
  Provider.of<EmailStore>(
    context,
    listen: false,
  ).onCompose = true;
  Navigator.of(context).push(
    PageRouteBuilder(
      pageBuilder: (
        BuildContext context,
        Animation<double> animation,
        Animation<double> secondaryAnimation,
      ) {
        return const ComposePage();
      },
    ),
  );
},

```

這會顯示如何前往電子郵件撰寫頁面，且不使用任何自訂轉場效果。在本程式碼研究室中，您將深入瞭解 `Reply` 的程式碼，設定與應用程式中各種導覽動作搭配運作的 `Material` 轉場效果。

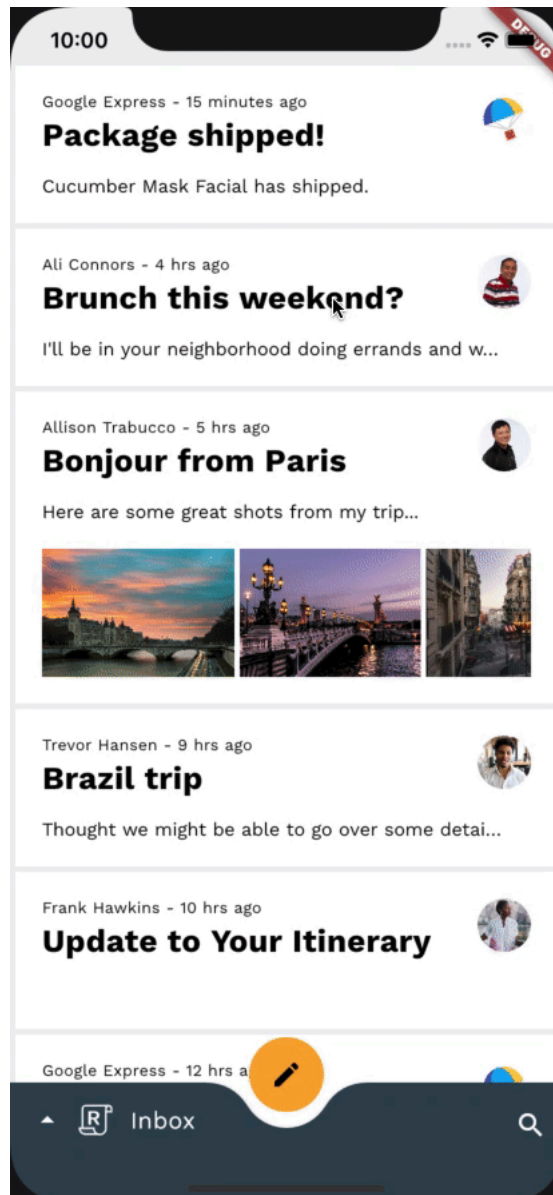
您已熟悉範例程式碼，接下來請實作第一個轉場效果。

5. 從電子郵件名單新增容器轉換轉場至電子郵件詳細資料頁面

首先，您要新增點選電子郵件時的轉場效果。容器轉換模式適用於這項導覽變更，因為這項模式是專為轉換含有容器的 UI 元素而設計。這個模式可在兩個 UI 元素之間建立起可見的連結。

加入任何程式碼前，請先嘗試執行 Reply 應用程式，然後點選電子郵件。這時應該會出現簡單的跳接，也就是畫面直接替換，沒有轉場效果：

變更前



動畫套件中的容器轉換稱為 `OpenContainer`。這個小工具的運作方式與 Flutter 中的任何其他小工具相同，因此您可以輕鬆將其插入應用程式，並運用它提供的複雜動畫。

首先，請在 `mail_card_preview.dart` 頂端新增動畫套件的匯入項目，如下列程式碼片段所示：

`mail_card_preview.dart`

```
import 'package:animations/animations.dart';
```

現在您已匯入動畫套件，可以開始為應用程式新增美觀的轉場效果。首先，請建立 `StatelessWidget` 類別，用來存放 `OpenContainer` 小工具。

在 `mail_card_preview.dart` 中，於 `MailPreviewCard` 的類別定義後方新增下列程式碼片段：

mail_card_preview.dart

```
// TODO: Add Container Transform transition from email list to email detail page (Motion)
class _OpenContainerWrapper extends StatelessWidget {
  const _OpenContainerWrapper({
    required this.id,
    required this.email,
    required this.closedChild,
  });

  final int id;
  final Email email;
  final Widget closedChild;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return OpenContainer(
      openBuilder: (context, closedContainer) {
        return MailViewPage(id: id, email: email);
      },
      openColor: theme.cardColor,
      closedShape: const RoundedRectangleBorder(
        borderRadius: BorderRadius.all(Radius.circular(0)),
      ),
      closedElevation: 0,
      closedColor: theme.cardColor,
      closedBuilder: (context, openContainer) {
        return InkWell(
          onTap: () {
            Provider.of<EmailStore>(
              context,
              listen: false,
            ).currentlySelectedEmailId = id;
            openContainer();
          },
          child: closedChild,
        );
      },
    );
  }
}
```

我們來分析一下 `_OpenContainerWrapper` 類別。從本質上來說，這是一個 `StatelessWidget`，會傳回包裝在 `OpenContainer` 小工具中的子項小工具 `closedChild`。 `OpenContainer` 小工具來自動畫套件，是開放容器轉換的來源。以下是部分屬性的用途：

- `openBuilder` 是在 `OpenContainer` 想要取得子項的開啟狀態時呼叫的函式。同樣地，`closedBuilder` 會對 `OpenContainer` 的關閉狀態執行相同操作。提供給兩個建構工具的動作 `Callback` 分別可用於開啟和關閉容器。我們使用提供給 `closedBuilder` 的動作 `Callback` 和 `openContainer()`，並將其傳遞至 `InkWell` 小工具的 `onTap`，因此輕觸 `InkWell` 即可觸發開啟容器。
- `openColor` 是容器開啟時的背景顏色，`closedColor` 則是容器關閉時的背景顏色。
- `closedShape` 是容器關閉時的形狀，同樣地，`openShape` 則定義容器開啟時的形狀。
- `closedElevation` 是容器關閉時的高度，`openElevation` 則是容器開啟時的高度。
- 在轉換期間，容器會在開啟和關閉的屬性之間轉換，並以流暢的動畫呈現。

現在來使用新的包裝函式。在 `MailPreviewCard` 類別定義中，我們會使用新的 `_OpenContainerWrapper` 包裝 `build()` 函式中的 `Material` 小工具：

mail_card_preview.dart

```
// TODO: Add Container Transform transition from email list to email detail page (Motion)
return _OpenContainerWrapper(
  id: id,
  email: email,
  closedChild: Material(
    ...
```

- 您會發現 `InkWell` 中 `onTap` 的邏輯與 `MailPreviewCard` 類別定義中定義的邏輯類似，但我們不再將路由推送至導覽器堆疊。 `OpenContainer` 會在內部處理路由，因此我們不必這麼做。
- 您可以使用 `Navigator.of(context).pop()` 觸發 `OpenContainer` 關閉。不過，請務必在正確的 `Navigator` 上呼叫 `pop()`，否則會發生非預期的行為。

我們的 `_OpenContainerWrapper` 具有 `InkWell` 小工具，而 `OpenContainer` 的顏色屬性會定義所封閉容器的顏色。因此，我們可以移除 `Material` 和 `InkWell` 小工具。完成的程式碼如下所示：

mail_card_preview.dart

```

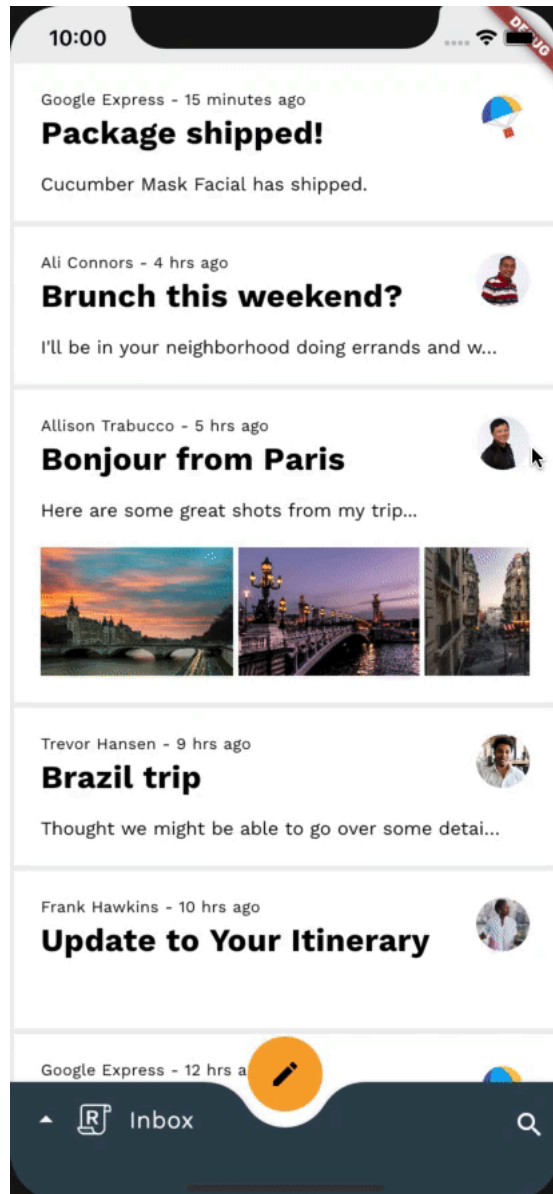
// TODO: Add Container Transform transition from email list to email detail page (Motion)
return _OpenContainerWrapper(
  id: id,
  email: email,
  closedChild: Dismissible(
    key: ObjectKey(email),
    dismissThresholds: const {
      DismissDirection.startToEnd: 0.8,
      DismissDirection.endToStart: 0.4,
    },
    onDismissed: (direction) {
      switch (direction) {
        case DismissDirection.endToStart:
          if (onStarredInbox) {
            onStar();
          }
          break;
        case DismissDirection.startToEnd:
          onDelete();
          break;
        default:
      }
    },
    background: _DismissibleContainer(
      icon: 'twotone_delete',
      backgroundColor: colorScheme.primary,
      iconColor: ReplyColors.blue50,
      alignment: Alignment.centerLeft,
      padding: const EdgeInsetsDirectional.only(start: 20),
    ),
    confirmDismiss: (direction) async {
      if (direction == DismissDirection.endToStart) {
        if (onStarredInbox) {
          return true;
        }
        onStar();
        return false;
      } else {
        return true;
      }
    },
    secondaryBackground: _DismissibleContainer(
      icon: 'twotone_star',
      backgroundColor: currentEmailStarred
        ? colorScheme.secondary
        : theme.scaffoldBackgroundColor,
      iconColor: currentEmailStarred
        ? colorScheme.onSecondary
        : colorScheme.onBackground,
      alignment: Alignment.centerRight,
      padding: const EdgeInsetsDirectional.only(end: 20),
    ),
  ),

```

```
child: mailPreview,  
) ,  
);
```

此時，您應該已擁有可正常運作的容器轉換。點選電子郵件後，清單項目會展開為詳細資料畫面，電子郵件清單則會縮小。按下返回鍵會將電子郵件詳細資料畫面收合回清單項目，同時放大電子郵件清單。

變更後

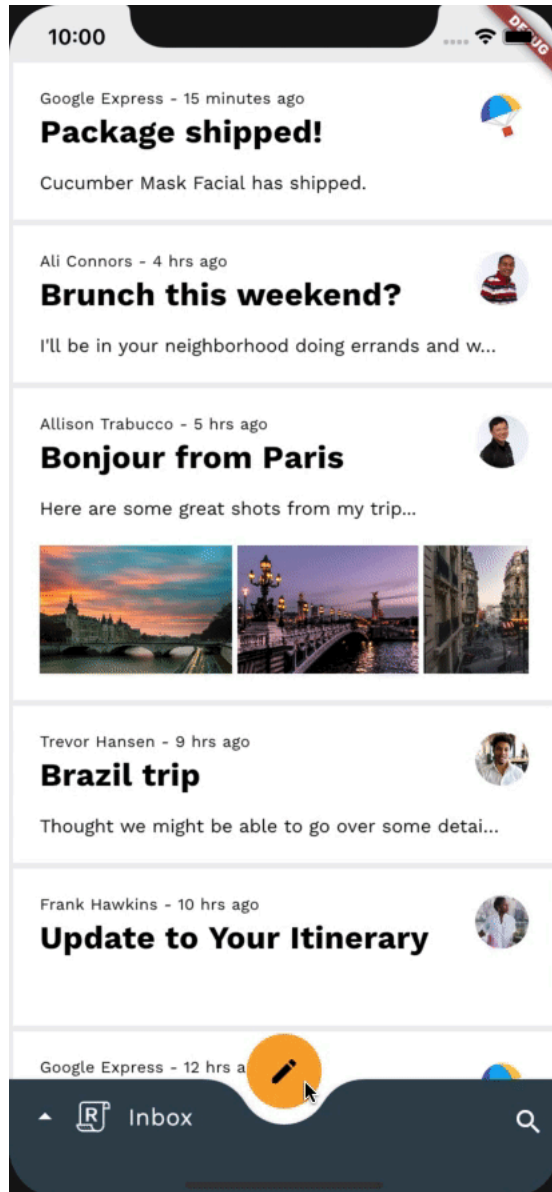


步驟 6 · 約 6 分鐘

6. 從 FAB 新增容器轉換轉場效果至撰寫電子郵件頁面

我們繼續使用容器轉換，並從懸浮動作按鈕新增轉場效果，將懸浮動作按鈕展開為使用者要撰寫的新電子郵件。ComposePage 首先，請重新執行應用程式，然後點選懸浮動作按鈕 (FAB)，確認啟動電子郵件撰寫畫面時沒有轉場效果。

變更前



由於我們使用相同的小工具類別 `OpenContainer`，因此設定這項轉場效果的方式與上一個步驟非常相似。

在 `home.dart` 中，於檔案頂端匯入 `package:animations/animations.dart`，並修改 `_ReplyFabState` `build()` 方法。讓我們使用 `OpenContainer` 小工具包裝傳回的 `Material` 小工具：

`home.dart`

```
// TODO: Add Container Transform from FAB to compose email page (Motion)
return OpenContainer(
  openBuilder: (context, closedContainer) {
    return const ComposePage();
  },
  openColor: theme.cardColor,
  onClose: (success) {
    Provider.of<EmailStore>(
      context,
      listen: false,
    ).onCompose = false;
  },
  closedShape: circleFabBorder,
  closedColor: theme.colorScheme.secondary,
  closedElevation: 6,
  closedBuilder: (context, openContainer) {
    return Material(
      color: theme.colorScheme.secondary,
      ...

```

除了用於設定先前 `OpenContainer` 小工具的參數外，現在也設定了 `onClosed`。`onClosed` 是 `ClosedCallback`，會在 `OpenContainer` 路由已彈出或返回關閉狀態時呼叫。該交易的傳回值會以引數形式傳遞至這個函式。我們會使用這個 `Callback` 通知應用程式的供應者，我們已離開 `ComposePage` 路由，以便通知所有事件監聽器。

與上一個步驟類似，我們會從小工具中移除 `Material` 小工具，因為 `OpenContainer` 小工具會處理 `closedBuilder` 傳回的小工具顏色 (使用 `closedColor`)。我們也會移除 `InkWell` 小工具 `onTap` 內的 `Navigator.push()` 呼叫，並以 `OpenContainer` 小工具 `closedBuilder` 提供的 `openContainer()` `Callback` 取代，因為現在 `OpenContainer` 小工具會處理自己的路由。

完成的程式碼如下：

[home.dart](#)

```
// TODO: Add Container Transform from FAB to compose email page (Motion)
return OpenContainer(
  openBuilder: (context, closedContainer) {
    return const ComposePage();
  },
  openColor: theme.cardColor,
  onClose: (success) {
    Provider.of<EmailStore>(
      context,
      listen: false,
    ).onCompose = false;
  },
  closedShape: circleFabBorder,
  closedColor: theme.colorScheme.secondary,
  closedElevation: 6,
  closedBuilder: (context, openContainer) {
    return Tooltip(
      message: tooltip,
      child: InkWell(
        customBorder: circleFabBorder,
        onTap: () {
          Provider.of<EmailStore>(
            context,
            listen: false,
          ).onCompose = true;
          openContainer();
        },
        child: SizedBox(
          height: _mobileFabDimension,
          width: _mobileFabDimension,
          child: Center(
            child: fabSwitcher,
          ),
        ),
      ),
    );
  },
);
```

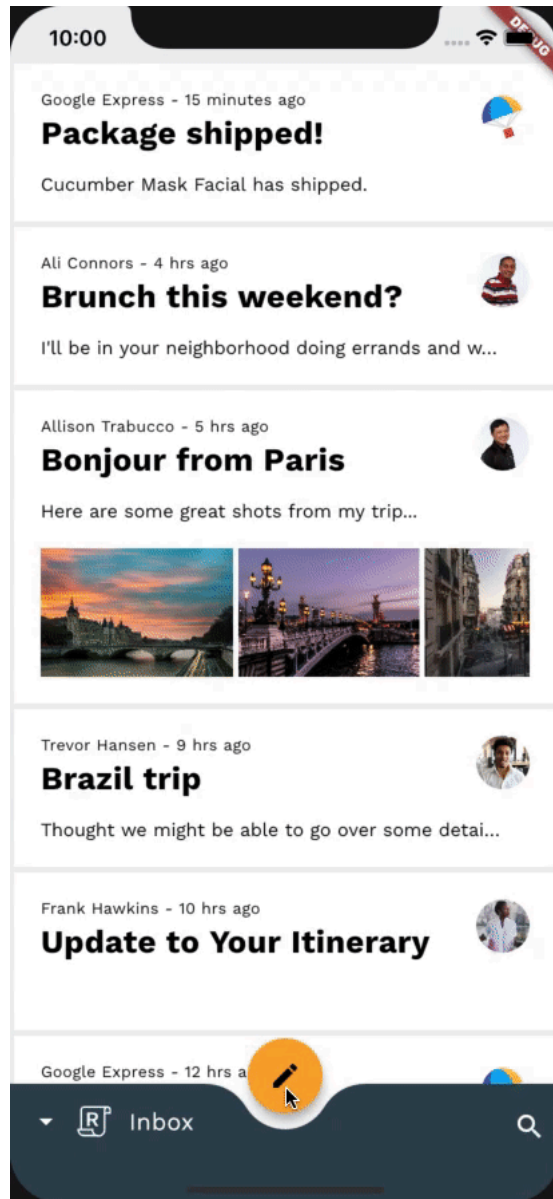
現在要清理一些舊程式碼。由於我們的 `OpenContainer` 小工具現在會透過 `onClosed` `ClosedCallback` 通知應用程式的供應商，我們已不再使用 `ComposePage`，因此可以移除 `mail_view_router.dart` 中的先前實作項目：

mail_view_router.dart

```
// TODO: Add Container Transform from FAB to compose email page (Motion)
emailStore.onCompose = false; /// delete this line
return SynchronousFuture<bool>(true);
```

這個步驟就完成了！您應該會看到從 FAB 到 Compose 畫面的轉場效果，如下所示：

變更後

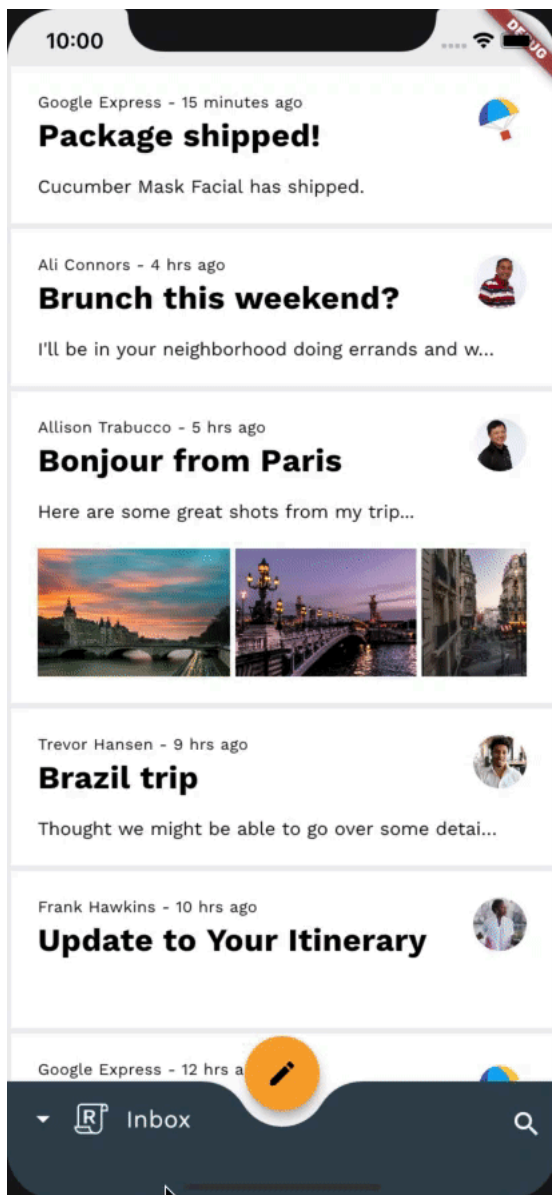


7. 從搜尋圖示到搜尋檢視畫面頁面，新增共用 Z 軸轉換

在這個步驟中，我們會新增從搜尋圖示到全螢幕搜尋檢視區塊的轉場效果。由於這項導覽變更不涉及任何持續性容器，我們可以運用「共用 Z 軸」轉場效果，強化兩個畫面之間的空間關係，並指出應用程式階層向上移動一個層級。

新增其他程式碼前，請先嘗試執行應用程式，然後輕觸畫面右下角的搜尋圖示。這時應該會顯示搜尋檢視畫面，且沒有任何轉場效果。

變更前



開始前，別忘了在要使用動畫套件的檔案頂端匯入 `package:animations/animations.dart`。

動畫套件中的共用軸轉換稱為 `SharedAxisTransition`。這個小工具具有下列屬性：

- `fillColor`：轉場期間使用的背景顏色。
- `animation`：用於驅動子項進入和離開的動畫。
- `secondaryAnimation`：在子項上方推送新內容時，用於轉場的動畫。
- `transitionType`：選擇縮放、水平和垂直類型的共用軸轉場效果。
- `child`：轉場效果的進入和退出小工具。

首先，請前往 `router.dart` 檔案。在 `ReplySearchPath` 類別定義後方新增下列程式碼片段：

router.dart

```
// TODO: Add Shared Z-Axis transition from search icon to search view page (Motion)
class SharedAxisTransitionPageWrapper extends Page {
  const SharedAxisTransitionPageWrapper(
    {required this.screen, required this.transitionKey})
    : super(key: transitionKey);

  final Widget screen;
  final ValueKey transitionKey;

  @override
  Route createRoute(BuildContext context) {
    return PageRouteBuilder(
      settings: this,
      transitionsBuilder: (context, animation, secondaryAnimation, child) {
        return SharedAxisTransition(
          fillColor: Theme.of(context).cardColor,
          animation: animation,
          secondaryAnimation: secondaryAnimation,
          transitionType: SharedAxisTransitionType.scaled,
          child: child,
        );
      },
      pageBuilder: (context, animation, secondaryAnimation) {
        return screen;
      });
  }
}
```

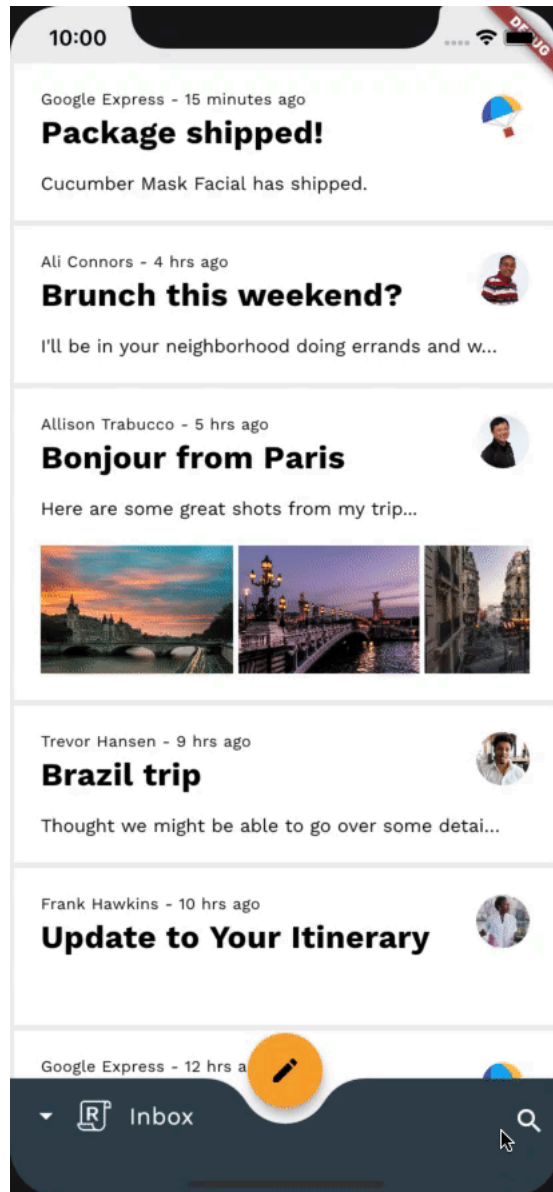
重要事項：如要在兩個元素之間轉換，請為每個元素指派不同的 `transitionKey`。如果沒有專屬 `transitionKey`，架構就無法區分兩者，因此不會顯示轉場效果。

現在，讓我們運用新的 `SharedAxisTransitionPageWrapper` 達成所需轉場效果。在 `ReplyRouterDelegate` 類別定義中，於 `pages` 屬性下方，使用 `SharedAxisTransitionPageWrapper` 包裝搜尋畫面，而非 `CustomTransitionPage`：

router.dart

```
return Navigator(  
  key: navigatorKey,  
  onPopPage: _handlePopPage,  
  pages: [  
    // TODO: Add Shared Z-Axis transition from search icon to search view page (Motion)  
    const CustomTransitionPage(  
      transitionKey: ValueKey('Home'),  
      screen: HomePage(),  
    ),  
    if (routePath is ReplySearchPath)  
      const SharedAxisTransitionPageWrapper(  
        transitionKey: ValueKey('Search'),  
        screen: SearchPage(),  
      ),  
  ],  
);
```

現在請嘗試重新執行應用程式。



一切看起來都很順利！當您點選底部應用程式列中的搜尋圖示時，共用軸轉場效果會將搜尋頁面縮放至檢視畫面。不過請注意，首頁不會縮放，而是保持靜態，搜尋頁面則會縮放並覆蓋首頁。此外，按下返回按鈕時，首頁不會縮放至可見範圍，而是會保持靜態，搜尋頁面則會縮放至可見範圍外。因此我們尚未完成。

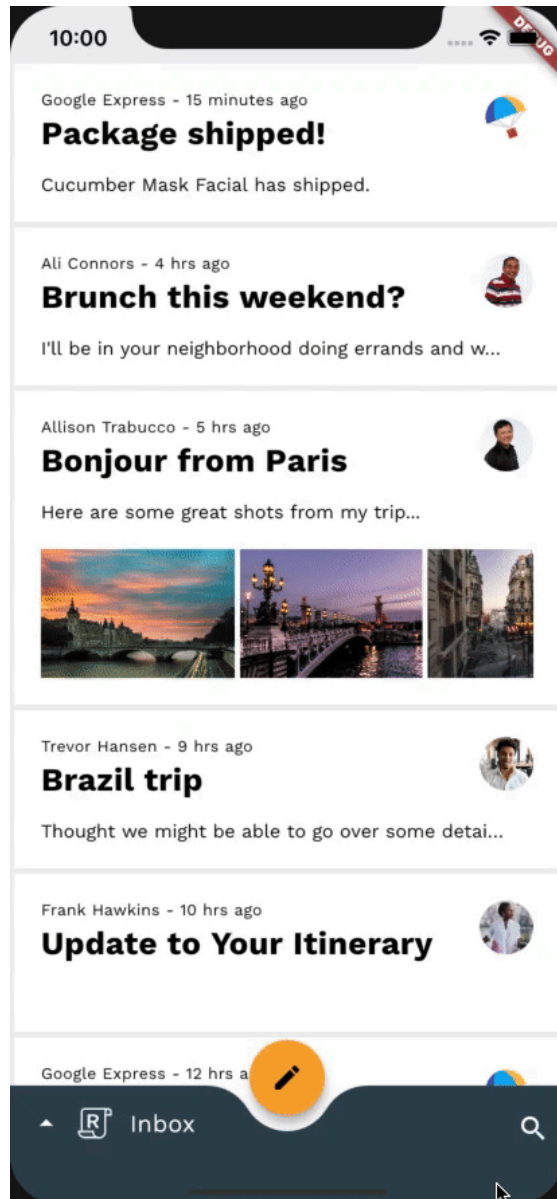
讓我們一併修正這兩個問題，方法是將 `HomePage` 包裝在 `SharedAxisTransitionWrapper` 中，而不是 `CustomTransitionPage`：

[router.dart](#)

```
return Navigator(  
  key: navigatorKey,  
  onPopPage: _handlePopPage,  
  pages: [  
    // TODO: Add Shared Z-Axis transition from search icon to search view page (Motion)  
    const SharedAxisTransitionPageWrapper(  
      transitionKey: ValueKey('home'),  
      screen: HomePage(),  
    ),  
    if (routePath is ReplySearchPath)  
      const SharedAxisTransitionPageWrapper(  
        transitionKey: ValueKey('search'),  
        screen: SearchPage(),  
      ),  
  ],  
);
```

大功告成！現在請嘗試重新執行應用程式，然後輕觸搜尋圖示。主畫面和搜尋畫面應同時淡出，並沿著 Z 軸縮放，在兩個畫面之間營造流暢效果。

變更後

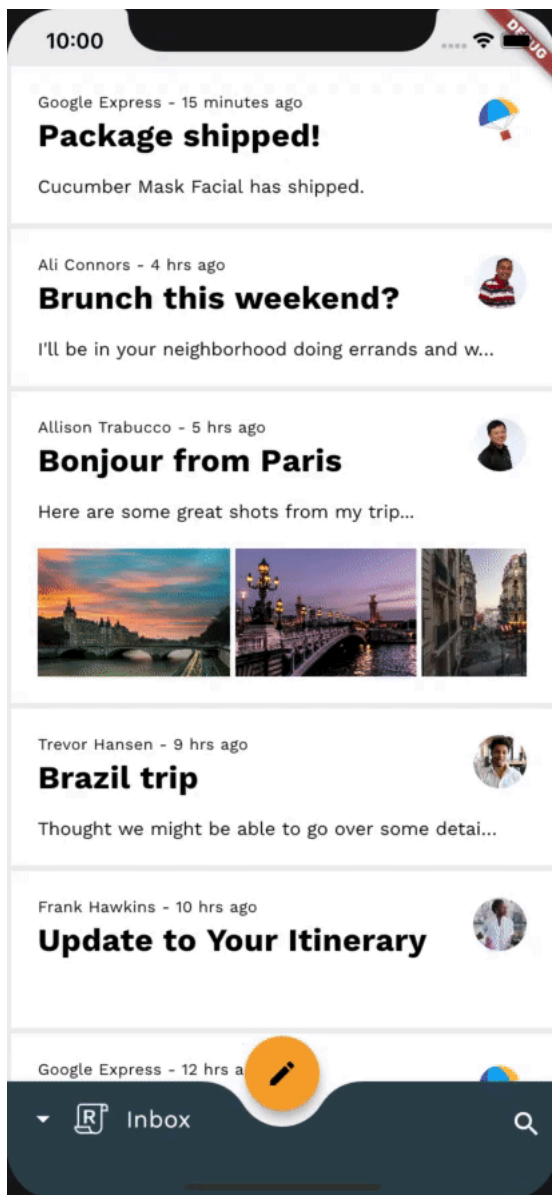


8. 在信箱頁面之間新增「淡出淡入」轉場效果

在這個步驟中，我們會在不同信箱之間新增轉場效果。由於我們不想強調空間或階層關係，因此會使用淡入淡出效果，在電子郵件清單之間執行簡單的「交換」作業。

在新增任何其他程式碼之前，請嘗試執行應用程式、輕觸底部應用程式列中的 Reply 標誌，然後切換信箱。電子郵件清單應會變更，且不會有轉場效果。

變更前



開始前，別忘了在要使用動畫套件的檔案頂端匯入 `package:animations/animations.dart`。

動畫套件中的淡入淡出轉場效果稱為 `FadeThroughTransition`。這個小工具具有下列屬性：

- `fillColor`：轉場期間使用的背景顏色。
- `animation`：用於驅動子項進入和離開的動畫。
- `secondaryAnimation`：在子項上方推送新內容時，用於轉場的動畫。
- `child`：轉場效果的進入和退出小工具。

首先，請前往 `mail_view_router.dart` 檔案。在 `MailViewRouterDelegate` 類別定義後方新增下列程式碼片段：

mail_view_router.dart

```
// TODO: Add Fade through transition between mailbox pages (Motion)
class FadeThroughTransitionPageWrapper extends Page {
  const FadeThroughTransitionPageWrapper({
    required this.mailbox,
    required this.transitionKey,
  }) : super(key: transitionKey);

  final Widget mailbox;
  final ValueKey transitionKey;

  @override
  Route createRoute(BuildContext context) {
    return PageRouteBuilder(
      settings: this,
      transitionsBuilder: (context, animation, secondaryAnimation, child) {
        return FadeThroughTransition(
          fillColor: Theme.of(context).scaffoldBackgroundColor,
          animation: animation,
          secondaryAnimation: secondaryAnimation,
          child: child,
        );
      },
      pageBuilder: (context, animation, secondaryAnimation) {
        return mailbox;
      }
    );
  }
}
```

與上一個步驟類似，我們將使用新的 `FadeThroughTransitionPageWrapper` 達成想要的轉場效果。在

`MailViewRouterDelegate` 類別定義中，於 `pages` 屬性下方，使用 `FadeThroughTransitionPageWrapper` 包裝信箱畫面，而非 `CustomTransitionPage`：

mail_view_router.dart

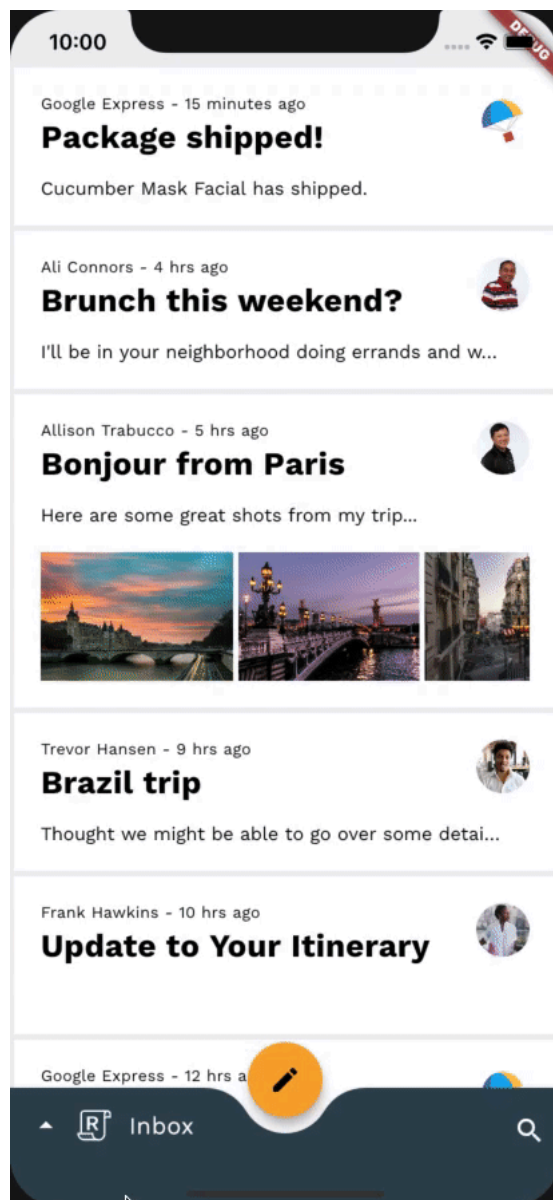
```

return Navigator(
  key: navigatorKey,
  onPopPage: _handlePopPage,
  pages: [
    // TODO: Add Fade through transition between mailbox pages (Motion)
    FadeThroughTransitionPageWrapper(
      mailbox: InboxPage(destination: currentlySelectedInbox),
      transitionKey: ValueKey(currentlySelectedInbox),
    ),
  ],
);

```

重新執行應用程式。開啟底部導覽匣並變更信箱時，目前的電子郵件清單應會淡出並縮放，而新清單則會淡入並縮放。太棒了！

變更後



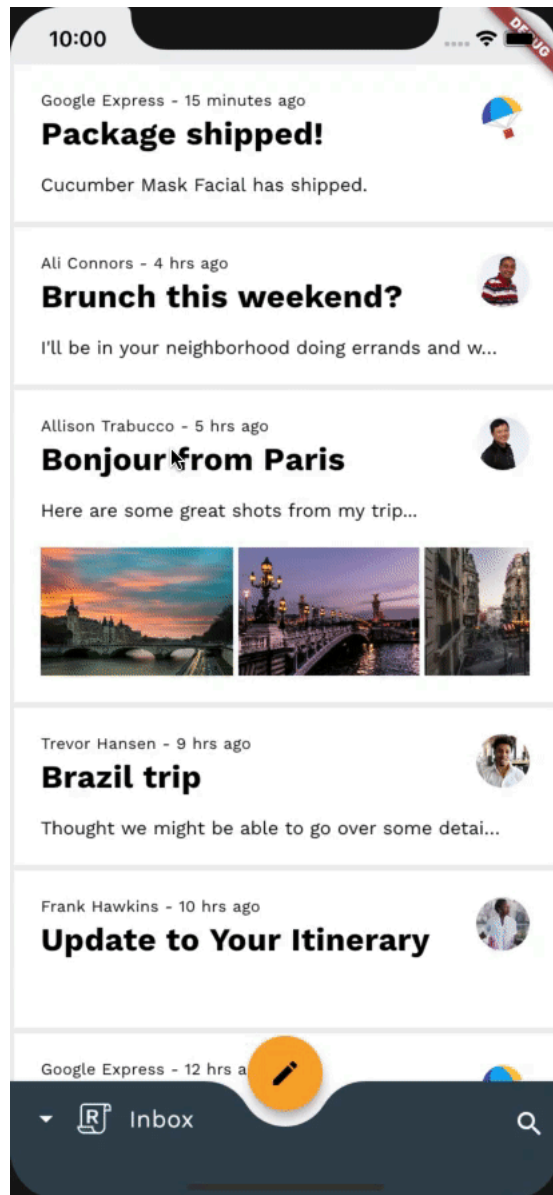
步驟 9 · 約 6 分鐘

9. 在撰寫和回覆懸浮動作按鈕之間新增「淡入/淡出」轉場效果

在這個步驟中，我們將在不同的 FAB 圖示之間新增轉場效果。由於我們不想強調空間或階層關係，因此會使用淡入淡出效果，在 FAB 中的圖示之間執行簡單的「交換」作業。

請先執行應用程式，輕觸電子郵件並開啟電子郵件檢視畫面，再新增任何其他程式碼。FAB 圖示應會變更，但不會有轉場效果。

變更前



在程式碼研究室的其餘部分，我們將使用 `home.dart`，因此不必擔心新增動畫套件的匯入項目，因為我們已在步驟 2 中為 `home.dart` 執行這項操作。

接下來幾項轉場效果的設定方式非常相似，因為這些效果都會使用可重複使用的類別 `_FadeThroughTransitionSwitcher`。

在 `home.dart` 中，於 `_ReplyFabState` 下方新增下列程式碼片段：

home.dart

```
// TODO: Add Fade through transition between compose and reply FAB (Motion)
class _FadeThroughTransitionSwitcher extends StatelessWidget {
  const _FadeThroughTransitionSwitcher({
    required this.fillColor,
    required this.child,
  });

  final Widget child;
  final Color fillColor;

  @override
  Widget build(BuildContext context) {
    return PageTransitionSwitcher(
      transitionBuilder: (child, animation, secondaryAnimation) {
        return FadeThroughTransition(
          fillColor: fillColor,
          child: child,
          animation: animation,
          secondaryAnimation: secondaryAnimation,
        );
      },
      child: child,
    );
  }
}
```

我們在最後一個步驟中已有一些使用 `FadeThroughTransition` 小工具的經驗。這個步驟會介紹動畫套件中的新小工具 `PageTransitionSwitcher`，每當子項使用 `transitionBuilder` 指定的動畫變更時，這個小工具就會從舊子項轉換為新子項。

- `transitionBuilder`：建構工具，可將新子項包裝在主要和次要動畫中，以決定子項進入和離開畫面的方式。我們會使用建構工具提供的 `animation` 和 `secondaryAnimation` 來驅動 `FadeThroughTransition`。
- 如果新舊子項都是同一種小工具，請記得為兩者提供專屬的 `key`，以便區分。

現在請在「`_ReplyFabState`」中尋找「`fabSwitcher`」小工具。`fabSwitcher` 會根據是否處於電子郵件檢視畫面，傳回不同的圖示。讓我們用 `_FadeThroughTransitionSwitcher` 納入：

[home.dart](#)

```
// TODO: Add Fade through transition between compose and reply FAB (Motion)
static final fabKey = UniqueKey();
static const double _mobileFabDimension = 56;

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final circleFabBorder = const CircleBorder();

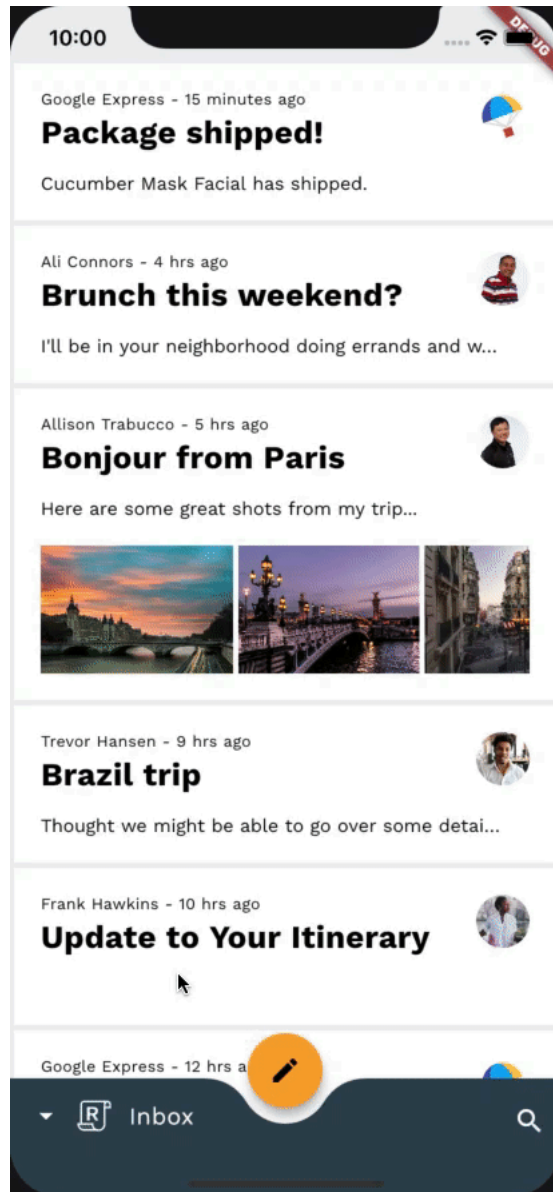
  return Selector<EmailStore, bool>(
    selector: (context, emailStore) => emailStore.onMailView,
    builder: (context, onMailView, child) {
      // TODO: Add Fade through transition between compose and reply FAB (Motion)
      final fabSwitcher = _FadeThroughTransitionSwitcher(
        fillColor: Colors.transparent,
        child: onMailView
          ? Icon(
              Icons.reply_all,
              key: fabKey,
              color: Colors.black,
            )
          : const Icon(
              Icons.create,
              color: Colors.black,
            ),
      );
    },
  );
  ...
}
```

我們為 `_FadeThroughTransitionSwitcher` 提供透明的 `fillColor`，因此元素之間不會有背景。我們也會建立 `UniqueKey`，並指派給其中一個圖示。

`UniqueKey` 可讓架構區分這兩個圖示。如果沒有專屬鍵，圖示之間就不會轉換，因為架構只會為小工具提供新參數，而不會重建小工具。

現在，您應該已擁有完整動畫效果的脈絡 FAB。進入電子郵件檢視畫面時，舊的 FAB 圖示會淡出並縮放，新的 FAB 圖示則會淡入並縮放。

變更後

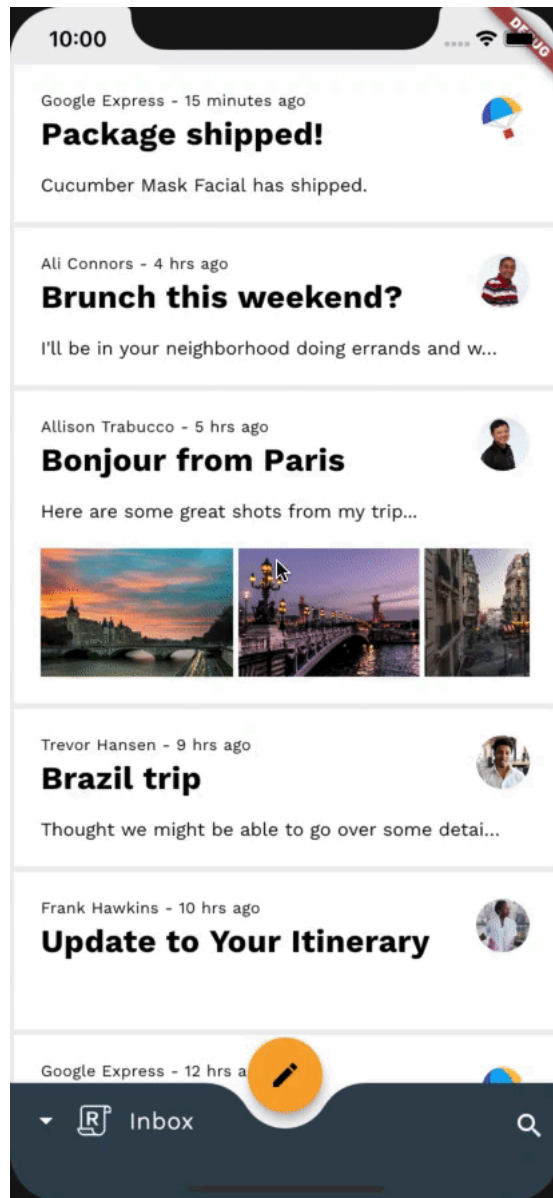


10. 在消失的信箱標題之間新增「淡出」轉場效果

在這個步驟中，我們會新增淡入淡出轉場效果，在電子郵件檢視畫面中，讓信箱標題在可見和不可見狀態之間淡入淡出。由於我們不想強調空間或階層關係，因此會使用淡入淡出效果，在包含信箱標題的 `Text` 小工具和空白 `SizedBox` 之間執行簡單的「交換」作業。

請先執行應用程式，輕觸電子郵件並開啟電子郵件檢視畫面，再新增任何其他程式碼。信箱標題應會立即消失。

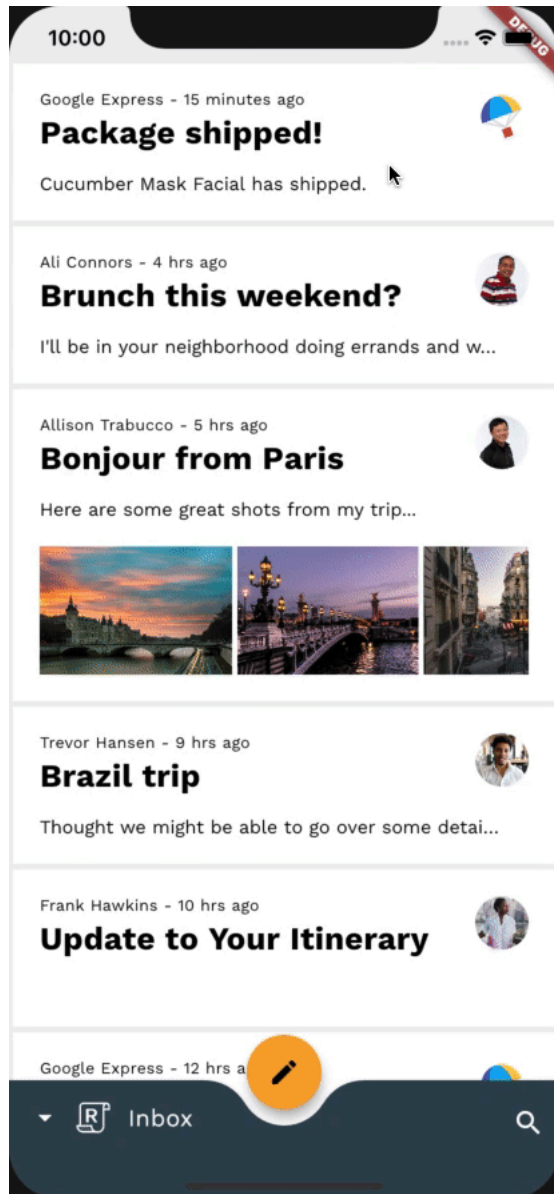
變更前



由於我們在最後一個步驟中已完成大部分工作，因此本程式碼研究室的其餘部分將快速完成。 `_FadeThroughTransitionSwitcher`

現在，請前往 `home.dart` 中的 `_AnimatedBottomAppBar` 類別，新增轉場效果。我們會重複使用上個步驟中的 `_FadeThroughTransitionSwitcher`，並包裝 `onMailView` 條件，傳回空白 `SizedBox` 或與底部導覽匣同步淡入的信箱標題：

[home.dart](#)

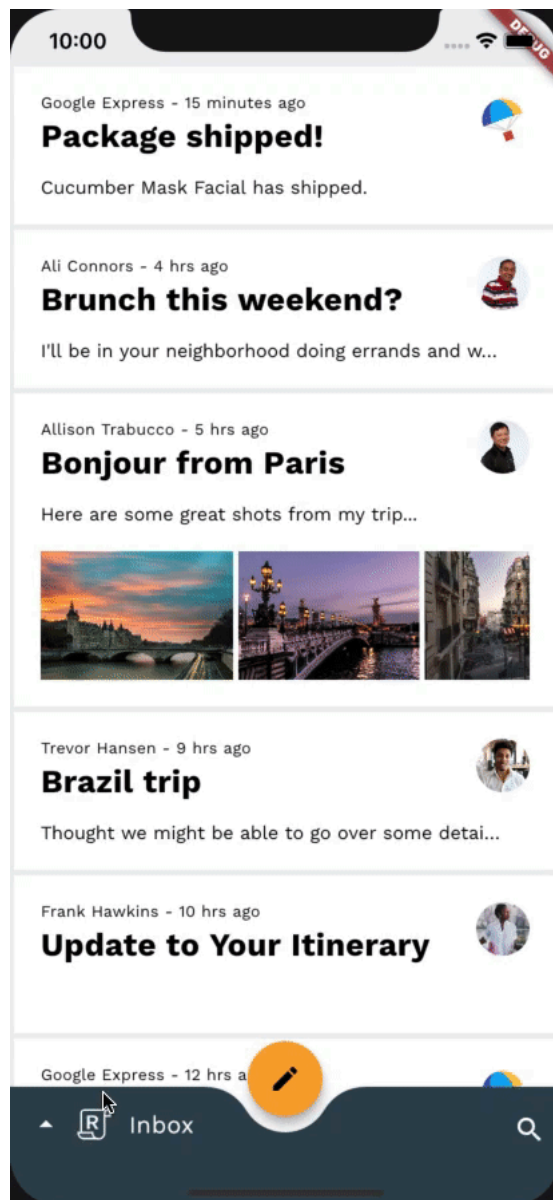


11. 在底部應用程式列動作之間新增「淡出淡入」轉場效果

在這個步驟中，我們會新增淡出淡入轉場效果，根據應用程式內容淡出淡入底部應用程式列動作。由於我們不想強調空間或階層關係，因此當應用程式位於首頁、底部導覽匣可見，以及我們位於電子郵件檢視畫面時，我們會使用淡入淡出效果，在底部應用程式列動作之間執行簡單的「交換」。

請先執行應用程式，輕觸電子郵件並開啟電子郵件檢視畫面，再新增任何其他程式碼。你也可以輕觸「回覆」標誌。底部應用程式列動作應會變更，且不會有轉場效果。

變更前



與上一個步驟類似，我們將再次使用 `_FadeThroughTransitionSwitcher`。如要達成所需轉場效果，請前往 `_BottomAppBarActionItems` 類別定義，並使用 `_FadeThroughTransitionSwitcher` 包裝 `build()` 函式的傳回小工具：

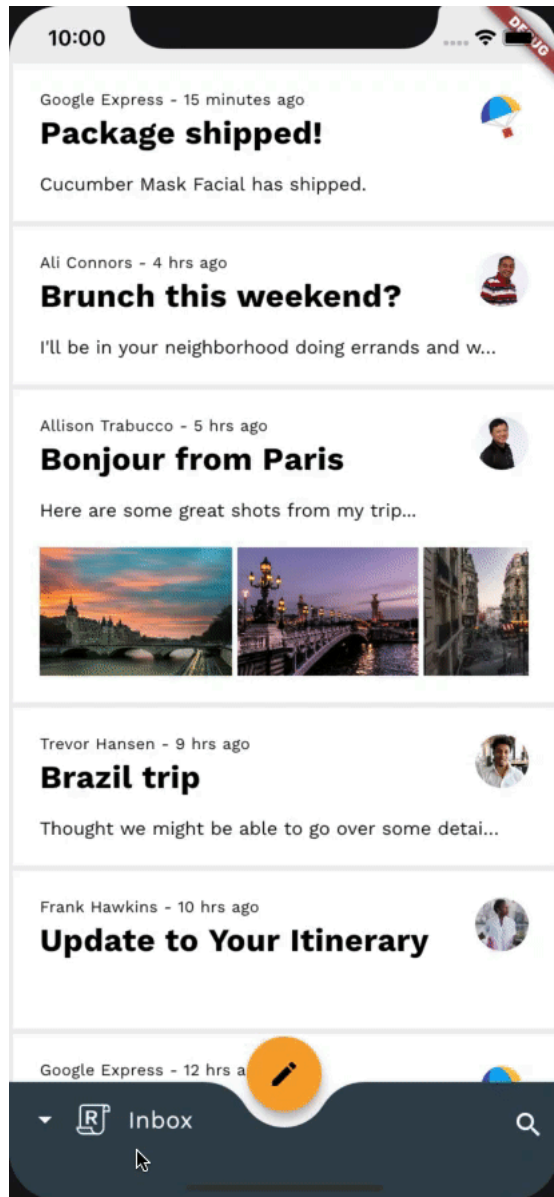
[home.dart](#)

```
// TODO: Add Fade through transition between bottom app bar actions (Motion)
return _FadeThroughTransitionSwitcher(
  fillColor: Colors.transparent,
  child: drawerVisible
    ? Align(
        key: UniqueKey(),
        alignment: AlignmentDirectional.bottomEnd,
        child: IconButton(
          icon: const Icon(Icons.settings),
          color: ReplyColors.white50,
          onPressed: () async {
            drawerController.reverse();
            showModalBottomSheet(
              context: context,
              shape: RoundedRectangleBorder(
                borderRadius: modalBorder,
              ),
              builder: (context) => const SettingsBottomSheet(),
            );
          },
        ),
      )
    : onMailView
  ...
```

我們也會在此為其中一個動作項目設定 `key` 參數。我們不會在每個待辦事項上設定，因為我們有兩個 `Align` 小工具和一個 `Row` 小工具。架構可以區分 `Row` 和 `Align` 小工具，但無法區分兩個 `Align` 小工具，因此我們為其中一個小工具設定了專屬鍵，以便在兩者之間轉換。

現在就來試試看吧！開啟電子郵件並進入電子郵件檢視畫面時，舊的底部應用程式列動作應會淡出並縮小，新的動作則會淡入並放大。非常好！

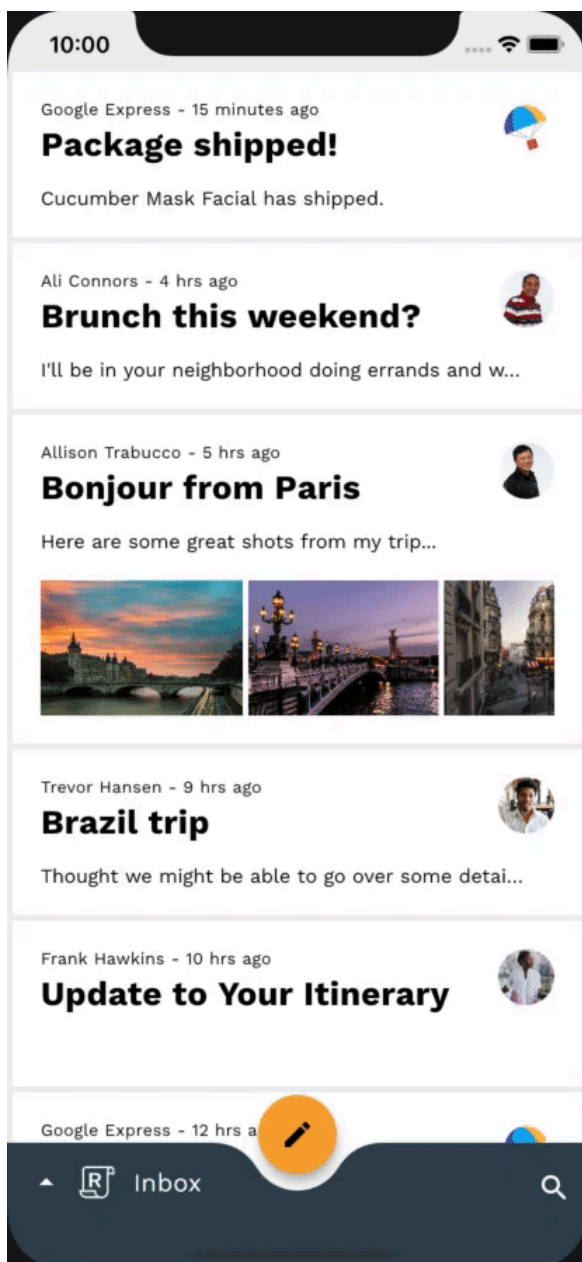
變更後



步驟 12 · 約 0 分鐘

12. 恭喜！

您使用不到 100 行的 Dart 程式碼，透過動畫套件在現有應用程式中建立美觀的轉場效果，不僅符合 Material Design 指南，在所有裝置上的外觀和行為也一致。



如要取得已完成的 Reply Material 動態系統應用程式，請下載[已完成的程式碼研究室](#)，或使用 Git 查看 `complete` 分支版本：`git checkout complete`。

如要查看本程式碼實驗室的所有程式碼變更，請查看 `complete` 分支版本的[差異比較](#)。

後續步驟

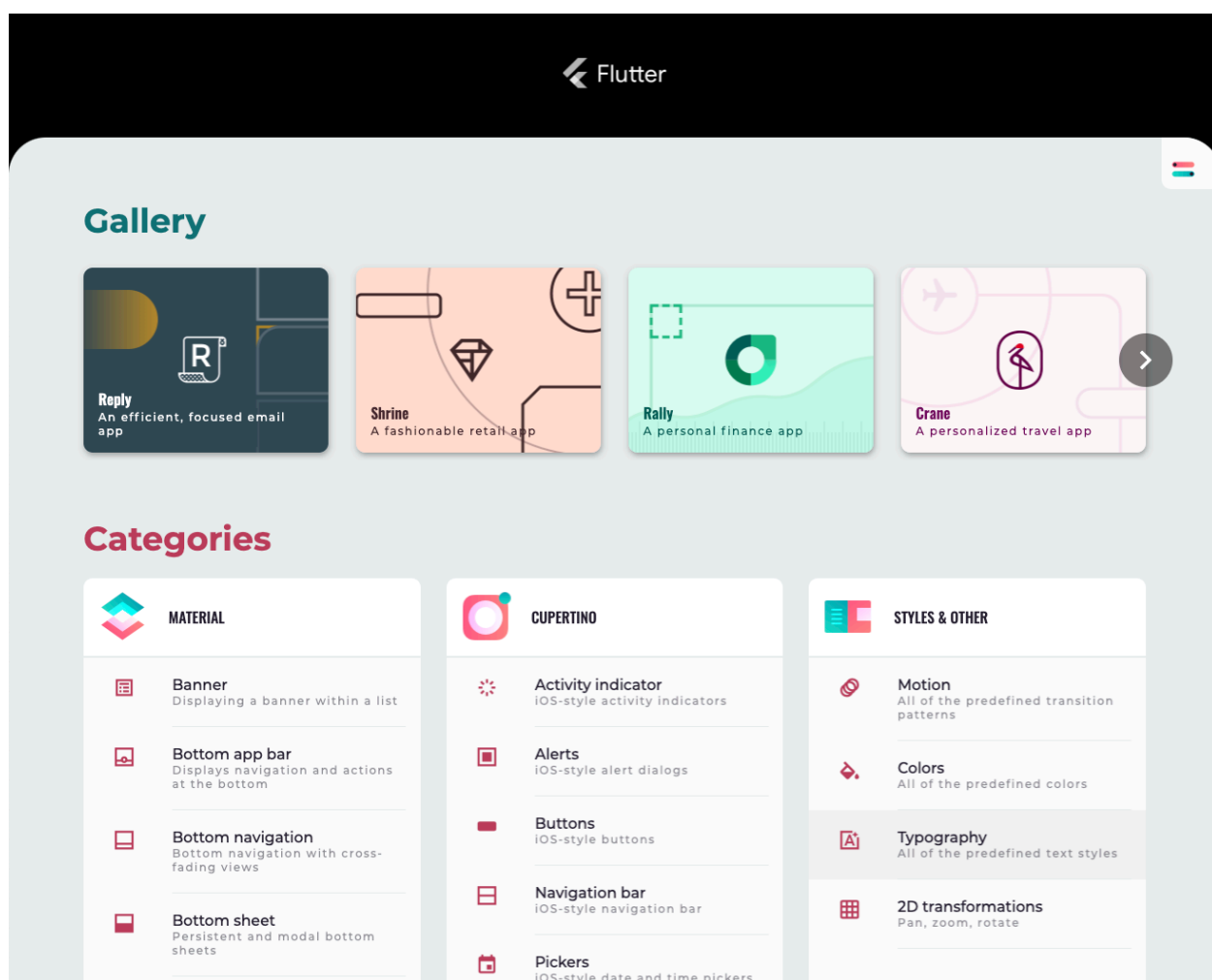
如要進一步瞭解 Material 動作系統，請務必參閱[指南](#)和完整的[開發人員說明文件](#)，並嘗試在應用程式中加入一些 Material 轉場效果！

感謝您試用 Material 動作。希望您喜歡這個程式碼研究室！

查看 Flutter Gallery



如要進一步瞭解如何使用 Material Flutter 程式庫提供的小工具，以及 Flutter 架構，請前往 [Flutter Gallery](#) 查看更多範例。



- [Material Motion 示範](#)
- [回覆材質研究](#)

Google Express - 15 minutes ago

Package shipped!

Cucumber Mask Facial has shipped.

☆

🗑

⋮



Ali Connors - 4 hrs ago

Brunch this weekend?

I'll be in your neighborhood doing errands and was hoping to catch you for a coffee this Saturday. If you don't have anything scheduled, it wo...

☆

🗑

⋮



Allison Trabucco - 5 hrs ago

Bonjour from Paris

Here are some great shots from my trip...



☆

🗑

⋮



Trevor Hansen - 9 hrs ago

Brazil trip

Thought we might be able to go over some details about our upcoming vacation.

☆

🗑

⋮



Frank Hawkins - 10 hrs ago

Update to Your Itinerary

☆

🗑

⋮



Google Express - 12 hrs ago

Delivered

Your shoes should be waiting for you at home!

☆

🗑

⋮

